

Bausteine Computergestützter Datenanalyse

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Franca Hollmann
Maik Poetzsch	Sebastian Seipel	

2026-01-20

Inhaltsverzeichnis

Methodenbaustein Datenfitting und Datenoptimierung	3
Prerequisites	4
Learning Objectives	5
1 Introduction	6
1.1 Plot	7
1.2 Code	7
1.3 Partial derivative with respect to the y-intercept	9
1.4 Partial derivative with respect to the slope	10
2 Interpolation – Filling in the Gaps	13
2.1 Overview	14
2.2 Polynomials	14
2.3 Polynomial Interpolation	18
3 Fitting	25
4 Splines	29
4.1 Definition	29
4.2 Cubic Splines	29
4.3 Application	30
5 Trend Smoothing – Reducing Noise	33
6 Exercises	35
6.1 Exercise: Analyzing Balloon Flight Data	35
6.2 Exercise: Beam Deflection in Civil Engineering	39
6.3 Exercise: Neutron Scattering	42

Methodenbaustein Datenfitting und Datenoptimierung



Bausteine Computergestützter Datenanalyse von Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch und Sebastian Seipel. Methodenbaustein Datenfitting und Datenoptimierung von Marc Fehr und Maik Poetzsch ist lizenziert unter [CC BY 4.0](#). Das Werk ist abrufbar auf [GitHub](#). Ausgenommen von der Lizenz sind alle Logos Dritter und anders gekennzeichneten Inhalte. 2025

Zitiervorschlag

Arnold, Lukas, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Franca Hollmann, Maik Poetzsch, und Sebastian Seipel. 2025. “Bausteine Computergestützter Datenanalyse. Methodenbaustein Datenfitting und Datenoptimierung”. <https://github.com/bausteine-der-datenanalyse/m-datenfitting-und-optimierung>.

BibTeX-Vorlage

```
@misc{BCD-m-datenfittung-und-optimierung-2025,  
  title={Bausteine Computergestützter Datenanalyse. Methodenbaustein Datenfitting und Datenop  
  author={Arnold, Lukas and Arnold, Simone and Bagemihl, Florian and Baitsch, Matthias and Fel  
  year={2025},  
  url={https://github.com/bausteine-der-datenanalyse/m-datenfitting-und-  
optimierung}}
```

Prerequisites

The estimated completion time for this module is approximately **placeholder**. The following modules are required to work on this unit:

- Python tool module
- NumPy tool module

The following modules and packages are used in this unit:

- numpy
 - numpy.polynomial
- matplotlib.pyplot

The unit uses the following data: No third-party data is used.

Learning Objectives

In this module, you will learn to...

- fill in missing data in a dataset with estimated values.
- describe data using a model function.
- describe data using multiple model functions.
- smooth noisy data.

1 Introduction

Experimentally obtained data can be heavily affected by noise, or the relationship between the variables may be best described by a non-linear dependence. Data fitting is the process of adapting a model to a dataset in order to describe the underlying relationship between the variables, smooth the data, or estimate values between the existing data points. The goal of this process is to find a function that describes the dataset as well as possible by minimizing the deviation between the model and the actual data. This process is also referred to as modeling.

In this module, the following packages are used:

```
import numpy as np
import numpy.polynomial.polynomial as poly
import matplotlib.pyplot as plt
```

The modeling of data can be generalized to the following problem:

1. Given are n pairs of measurement points (x_i, y_i) with $x_i, y_i \in \mathbb{R}$.
2. The goal is to find a model function $y(x)$ that approximates the measurement point pairs.

One possible approach is to represent the model function as a sum of m basis functions $\phi_i(x)$ with coefficients β_i .

$$y(x) = \sum_{i=1}^m \beta_i \cdot \phi_i(x) = \beta_1 \cdot \phi_1(x) + \dots + \beta_m \cdot \phi_m(x)$$

The coefficients β_i must be determined in such a way that $y(x)$ approximates the measurement points as well as possible – or even exactly.

As a distance measure between a model function and the measurement points, the method of least squares can be used.

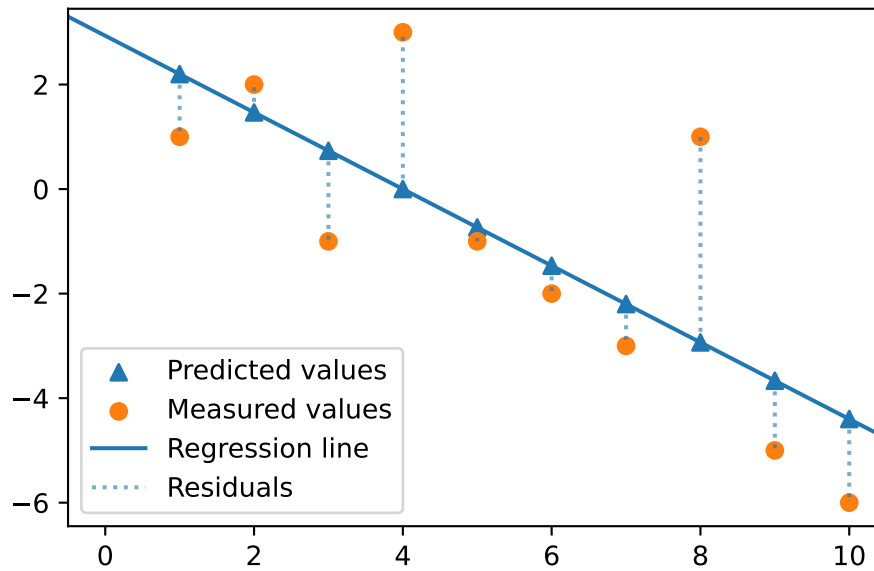
i Hinweis 1: Method of Least Squares

Using the method of least squares, the line $\hat{y} = \beta_0 + \beta_1 \cdot x$ is determined such that the squared distances of the predicted values $\hat{y}$ from the actually measured values y are minimized. The values $y_i - \hat{y}_i$ are the residuals e_i . Thus, the following holds:

$$\sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N e_i^2 = \min$$

Graphically, the minimization of the squared distances can be illustrated as follows.

1.1 Plot



Regression coefficients: [2.93333333 -0.73333333]

1.2 Code

```

x = np.arange(1, 11)
y = -x.copy() + 4
y[0] -= 2
y[2] -= 2
y[3] += 3
y[-3] += 5

lm = poly.polyfit(x, y, 1)
predicted_values = poly.polyval(x, lm)

plt.scatter(x, predicted_values, label='Predicted values', marker="^", color="tab:blue")
plt.scatter(x, y, label='Observed values', marker='o', color="tab:orange")
plt.axline(xy1=(0, lm[0]), slope=lm[1], label="Regression line", color="tab:blue")
dotted = plt.vlines(x, ymin=predicted_values, ymax=y, alpha=0.6, ls='dotted', label='Residuals')

plt.legend()
plt.show()

print("Regression coefficients:", lm)

```

The drawn straight line corresponds to the linear function $\hat{y} = \beta_0 + \beta_1 \cdot x + e_i$. The triangular markers represent the predicted values $\hat{y}_i$ of the linear model for the values $x_i = \text{np.arange}(1, 11)$. The actual observed values y are marked with circular markers. The length of the dashed lines corresponds to the magnitude of the deviation between the observed and predicted values $y_i - \hat{y}_i$, i.e. the residuals e_i . The goal is to find the straight line that minimizes the sum of the squared residuals. The desired values β_0 and β_1 are the **least-squares estimators**.

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

For completeness, we derive the least-squares estimators. We are looking for values of β_0 and β_1 such that the sum of squared residuals $\sum_{i=1}^n e_i^2$ is as small as possible. The sum of squared residuals is the sum of the squared differences between the observed values y_i and the values predicted by the linear function.

$$\sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i))^2$$

We therefore investigate a function that depends on two variables.

$$f(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i))^2$$

The summation symbol is shorthand notation for a sum.

$$f(\beta_0, \beta_1) = (y_1 - (\beta_0 + \beta_1 \cdot x_1))^2 + (y_2 - (\beta_0 + \beta_1 \cdot x_2))^2 + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n))^2$$

At the minimum of the function, both partial derivatives must be equal to zero (why this is the case is explained clearly [here](#).)

Beispiel 1.1: Partial derivative

A partial derivative is the derivative of a function with several variables with respect to one variable, while the other variables are treated as constants.

For a function $f(x, y) = 2x + y^2$, the partial derivative with respect to x is written as:

$$\frac{\partial f(x, y)}{\partial x}$$

- The symbol ∂ is the cursive form of the Cyrillic lowercase letter ∂ (d) and is read as “del”. It indicates that a partial derivative is taken.
- The numerator contains the function to be differentiated. The denominator contains the variable with respect to which the differentiation is performed. The expression is read as “del f of x and y with respect to del x”.

The partial derivative

$$\frac{\partial f(x, y)}{\partial x} = 2.$$

The term y^2 is treated as a constant (e.g. 5^2) and its derivative is zero.

The partial derivative

$$\frac{\partial f(x, y)}{\partial y} = 2y.$$

The term $2x$ is treated as a constant (e.g. $2 \cdot 3$) and its derivative is zero.

In both partial derivatives, x_i and y_i are constants.

In the partial derivative with respect to β_0 , β_1 is also constant;

in the partial derivative with respect to β_1 , β_0 is constant accordingly.

1.3 Partial derivative with respect to the y-intercept

For the partial derivative with respect to β_0 , the chain rule applies to the outer function (top) and the inner function (middle):

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_0} = 2 \cdot (y_1 - (\beta_0 + \beta_1 \cdot x_1)) + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n)) = 2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) \cdot (0 - (1 + 0 \cdot 0))$$

Thus, for the partial derivative with respect to β_0 :

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_0} = -2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) = 0$$

This expression can be simplified by dropping the prefactor -2 (since $-2 \cdot 0 = 0$) and resolving the signs:

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 \cdot x_i) = 0$$

This can also be written as:

$$\sum_{i=1}^n y_i - \sum_{i=1}^n \beta_0 - \sum_{i=1}^n \beta_1 \cdot x_i = 0$$

β_0 and β_1 are constants, so

$$\sum_{i=1}^n \beta_0 = \beta_0 \cdot \sum_{i=1}^n 1 = \beta_0 \cdot n \text{ and}$$

$$\sum_{i=1}^n \beta_1 \cdot x_i = \beta_1 \cdot \sum_{i=1}^n 1 \cdot x_i.$$

Thus:

$$\sum_{i=1}^n y_i - n \cdot \beta_0 - \beta_1 \cdot \sum_{i=1}^n x_i = 0$$

Dividing by n , we obtain the arithmetic means

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i \text{ and}$$

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \text{ yielding:}$$

$$\bar{y} - \beta_0 - \beta_1 \cdot \bar{x} = 0$$

Rearranging gives:

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

1.4 Partial derivative with respect to the slope

For the partial derivative with respect to β_1 , the chain rule is applied again, so the outer function is differentiated in the same way:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_1} = 2 \cdot (y_1 - (\beta_0 + \beta_1 \cdot x_1)) + \dots (y_n - (\beta_0 + \beta_1 \cdot x_n)) = 2 \cdot \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \cdot x_i)) \cdot (0 - (0 + 1 \cdot x_i))$$

Thus:

$$\frac{\partial f(\beta_0, \beta_1)}{\partial \beta_1} = -2 \sum_{i=1}^n x_i \cdot (y_i - (\beta_0 + \beta_1 \cdot x_i)) = 0$$

This can also be simplified by omitting the prefactor -2 (since $-2 \cdot 0 = 0$ must hold) and resolving the signs. In addition, the expression can be expanded:

$$\sum_{i=1}^n x_i y_i - \sum_{i=1}^n \beta_0 \cdot x_i - \sum_{i=1}^n \beta_1 \cdot x_i x_i = 0$$

Extracting constants:

$$\sum_{i=1}^n x_i y_i - \beta_0 \cdot \sum_{i=1}^n x_i - \beta_1 \cdot \sum_{i=1}^n x_i x_i = 0$$

Substituting $\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$ and $\sum_{i=1}^n x_i = n \cdot \bar{x}$:

$$\sum_{i=1}^n x_i y_i - (\bar{y} - \beta_1 \cdot \bar{x}) \cdot n \cdot \bar{x} - \beta_1 \cdot \sum_{i=1}^n x_i x_i = 0$$

Expanding the middle term and writing $x_i x_i$ as x_i^2 :

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - \beta_1 \cdot n \bar{x} \bar{x} - \beta_1 \cdot \sum_{i=1}^n x_i^2 = 0$$

Combining the last two terms:

$$\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y} - \beta_1 \cdot (\sum_{i=1}^n x_i^2 - n \bar{x}^2) = 0$$

Solving for β_1 :

$$\beta_1 \cdot (\sum_{i=1}^n x_i^2 - n \bar{x}^2) = \sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}$$

Then:

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2}$$

Now, one can first rewrite using $\sum_{i=1}^n x_i^2 - n \bar{x}^2 = \sum_{i=1}^n (x_i - \bar{x})^2$.

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Then—and this will be shown next—using $\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$. Thus, we obtain:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

The last step is shown starting from the result and begins with expanding the product:

$$\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n (x_i y_i - x_i \bar{y} - \bar{x} y_i + \bar{x} \bar{y})$$

This can also be written as:

$$\sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \bar{y} - \sum_{i=1}^n \bar{x} y_i + \sum_{i=1}^n \bar{x} \bar{y}$$

$\bar{x}$ and $\bar{y}$ are constants, so this can be written as $\bar{x} \cdot \sum_{i=1}^n y_i$ and $\bar{y} \cdot \sum_{i=1}^n x_i$. Since $\sum_{i=1}^n x_i = n \cdot \bar{x}$ (analogously for y), we obtain:

$$\sum_{i=1}^n x_i y_i - \bar{y} \cdot n \cdot \bar{x} - \bar{x} \cdot n \cdot \bar{y} + \sum_{i=1}^n \bar{x} \bar{y}$$

Rearranging:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} - n\bar{x}\bar{y} + \sum_{i=1}^n \bar{x} \bar{y}$$

The last term $\sum_{i=1}^n \bar{x} \bar{y}$ can also be written as $n \cdot \bar{x} \bar{y}$, which yields:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} - n\bar{x}\bar{y} + n\bar{x}\bar{y}$$

Thus, the last two terms cancel out, leaving:

$$\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y}$$

(Baitsch, Matthias [2019](#), pp. 73–74)

In this chapter, the following methods for data modeling are introduced:

1. Polynomial interpolation
2. Data fitting using polynomials
3. Data fitting using splines

2 Interpolation – Filling in the Gaps

What to do when values are missing? In many datasets, there are gaps – for example, because measurements were only taken at specific points. **Interpolation** is a method that allows us to estimate intermediate values, i.e., values within a known range.

In contrast, **extrapolation** attempts to predict values outside the known range – which is generally associated with greater uncertainty.

In interpolation, we look for a model function that exactly represents the measured data.

Theory – Modeling

The goal of data modeling is to represent a set of data through a functional relationship. Data from experiments or simulations can be heavily noisy and unsuitable for further processing. A smoothing function can greatly simplify the dataset. Alternatively, if there are only a few data points, the intermediate values must be determined by a function. In general, data modeling can be formalized as the following problem:

1. Given are n measurement point pairs (x_i, y_i) with $x_i, y_i \in \mathbb{R}$
2. Find a model function $y(x)$ that approximates the measurement point pairs.

One possible approach is to represent the model function as a sum of m basis functions $\phi_i(x)$ with coefficients β_i :

$$y(x) = \sum_{i=1}^m \beta_i \cdot \phi_i(x) = \beta_1 \cdot \phi_1(x) + \dots + \beta_m \cdot \phi_m(x)$$

The coefficients β_i must be determined such that $y(x)$ approximates the measurement points as closely as possible – or even exactly.

As a measure of distance between a model function and the data points, the [L2 norm](#) can be used. It is defined as

$$\|y(x) - (x_i, y_i)\|_2 = \sum_{i=1}^n (y(x_i) - y_i)^2 \quad .$$

Such a norm provides a measure of the quality of an approximation: the smaller the distance, the better the quality. This allows for finding optimal coefficients and is, for

example, used in the **least squares method**, where a set of coefficients is sought that minimizes the L2 norm.

2.1 Overview

In many practical applications, polynomials are assumed as basis functions for the model function. Advantages of polynomials:

- Polynomials are easy to differentiate and integrate.
- Any function can be approximated using polynomials, see [Taylor series](#).
- Evaluation requires only a few arithmetic operations (addition and multiplication), making them simple and fast to use.

An example of a polynomial basis:

$$\phi_1(x) = 1, \quad \phi_2(x) = x, \quad \phi_3(x) = x^2, \quad \dots, \quad \phi_m = x^{m-1}$$

2.2 Polynomials

Polynomials $P(x)$ are functions in the form of a sum of power functions with natural number exponents ($x^i, i \in \mathbb{N}$) and corresponding coefficients a_i :

$$P(x) = \sum_{i=0}^n a_i x^i = a_n x^n + a_{n-1} x^{n-1} + \dots + a_2 x^2 + a_1 x + a_0, \quad i, n \in \mathbb{N}, a_i \in \mathbb{R}$$

- The **degree** of a polynomial is the term with the highest exponent that has a non-zero coefficient (the so-called leading coefficient).
- A polynomial of degree n has n roots, which can be [complex](#).

In the NumPy module, polynomials are represented by their coefficients. Generally, a polynomial of degree n is represented by the following array:

```
[a0, a1, a2, ..., an]
```

For example, for $P(x) = 3 - 2x + 5x^2 + x^3$:

```
P = np.array([3, -2, 5, 1])  
print(P)
```

```
[ 3 -2  5  1]
```

Functions for evaluating polynomial functions are provided by the `numpy.polynomial` package, specifically its module `numpy.polynomial.polynomial`.

```
import numpy.polynomial.polynomial as poly
```

The evaluation of a polynomial at a point or an array is done using the `poly.polyval(x, c)` function. This calculates the function values for the values passed in `x` using the function parameters `c`.

```
x = 1
y = poly.polyval(x, P)
print(f"P(x={x}) = {y}")
```

$P(x=1) = 7.0$

```
x = np.array([-1, 0, 1])
y = poly.polyval(x, P)
print(f"P(x={x}) = {y}")
```

$P(x=[-1 \ 0 \ 1]) = [9. \ 3. \ 7.]$

The Matplotlib package can be used for graphical representation in the range $x \in [-6, 2]$.

```
# Generate 50 points between -6 and 2

x = np.linspace(-6, 2, 50)

# Evaluate the polynomial P at points x

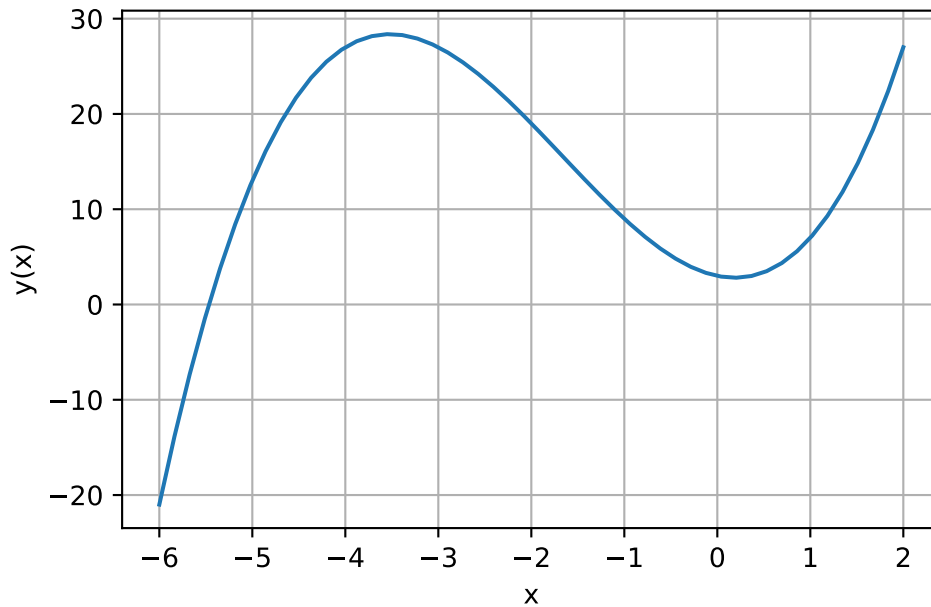
y = poly.polyval(x, P)

# Plot the polynomial

plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('y(x)')
plt.grid()
```

```
# Display the plot
```

```
plt.show()
```



To find the roots of a polynomial, the function `poly.polyroots()` can be used. For the polynomial above, the following roots can be determined.

```
roots = poly.polyroots(P)
```

```
# direct output of the array
```

```
print("Roots: ")
```

```
print(roots)
```

Roots:

```
[-5.46628038+0.j          0.23314019-0.703182j  0.23314019+0.703182j]
```

```
print("Roots: ")
```

```
# nicer display of the array
```

```

for i, root in enumerate(roots):
    if root.imag == 0:
        print(f" x_{i+1} = {root.real:.2}")
    else:
        print(f" x_{i+1} = {root.real:.2} {root.imag:+.2}i")

```

Roots:

```

x_1 = -5.5
x_2 = 0.23 -0.7i
x_3 = 0.23 +0.7i

```

In this example, two of the roots are complex. A complex number z in Python is represented as the sum of its real part (Re) and imaginary part (Im). The latter is indicated by a trailing j , the imaginary unit.

$$z = Re(z) + Im(z)j$$

The roots can also be used to represent the polynomial alternatively. If x_i are the n roots, then the n -th degree polynomial is described by the following product:

$$P(x) = \prod_{i=1}^n (x - x_i) = (x - x_1) \cdot (x - x_2) \cdot \dots \cdot (x - x_n)$$

For example, if 1 and 2 are the roots of a polynomial, then it is:

$$P(x) = (x - 1)(x - 2) = 2 - 3x + x^2$$

The function `poly.polyfromroots(roots)` can determine the polynomial coefficients from the roots. Based on the example above, the function call would be:

```

roots = [1, 2]
coefficients = poly.polyfromroots(roots)

print("Roots:", roots)
print("Coefficients:", coefficients)

```

```

Roots: [1, 2]
Coefficients: [ 2. -3.  1.]

```

The module `numpy.polynomial.polynomial` provides many convenient functions for working with polynomials. There are also functions for adding, multiplying, differentiating, or integrating polynomials. An overview can be found in the [NumPy documentation](#).

2.3 Polynomial Interpolation

Interpolation is a method to construct data points between given measurements. A function is sought that exactly matches all measurement points, which is equivalent to the L2-norm between the function and the points being zero. For example, two points can be interpolated with a straight line.

That is, for two measurement pairs (x_1, y_1) and (x_2, y_2) with $x_1 \neq x_2$, there exists a set of coefficients such that the L2-norm between the points and the model function

$$y(x) = \beta_0 + \beta_1 x$$

vanishes.

```
# Example data from y(x) = 2 - x

N = 50
dx = 0.25

def func(x):
    return 2 - x

x = np.array([1, 2])
y = func(x)

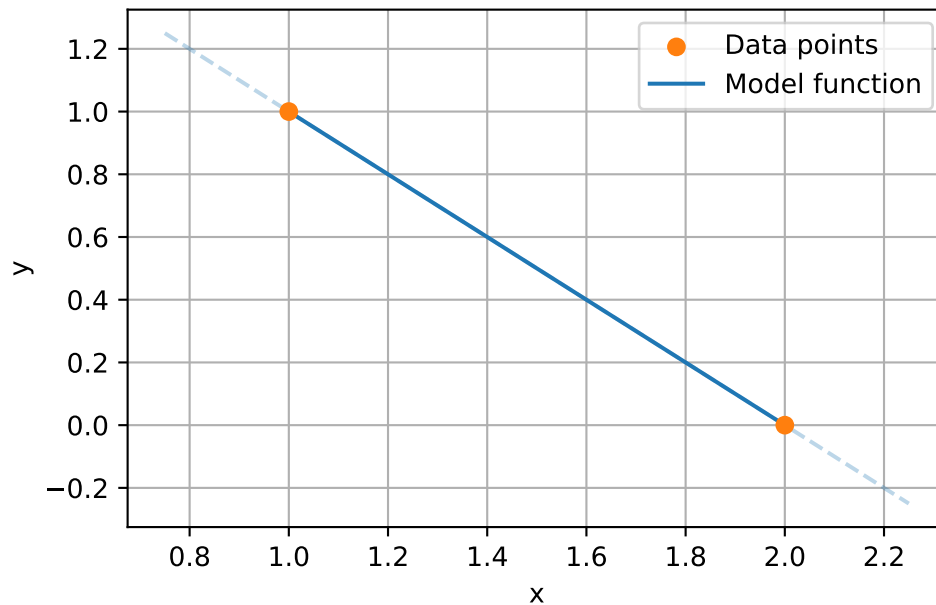
plt.scatter(x, y, color='C1', label="Data points", zorder=3)

x_model = np.linspace(np.min(x), np.max(x), N)
plt.plot(x_model, func(x_model), color='C0', label="Model function")

x_line = np.linspace(np.min(x)-dx, np.max(x)+dx, N)
plt.plot(x_line, func(x_line), '--', alpha=0.3, color='C0')

plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
```

```
plt.show()
```



For three data points, a second-degree polynomial is used to exactly capture the points.

$$y(x) = \beta_0 + \beta_1 x + \beta_2 x^2$$

```
# Example data from  $y(x) = -1 - 4x + 3x^2$ 

N = 50
dx = 0.25

def func(x):
    return -1 - 4*x + 3*x**2

x = np.array([-1, 2, 3])
y = func(x)

plt.scatter(x, y, color='C1', label="Data points", zorder=3)

x_model = np.linspace(np.min(x), np.max(x), N)
plt.plot(x_model, func(x_model), color='C0', label="Model function")
```

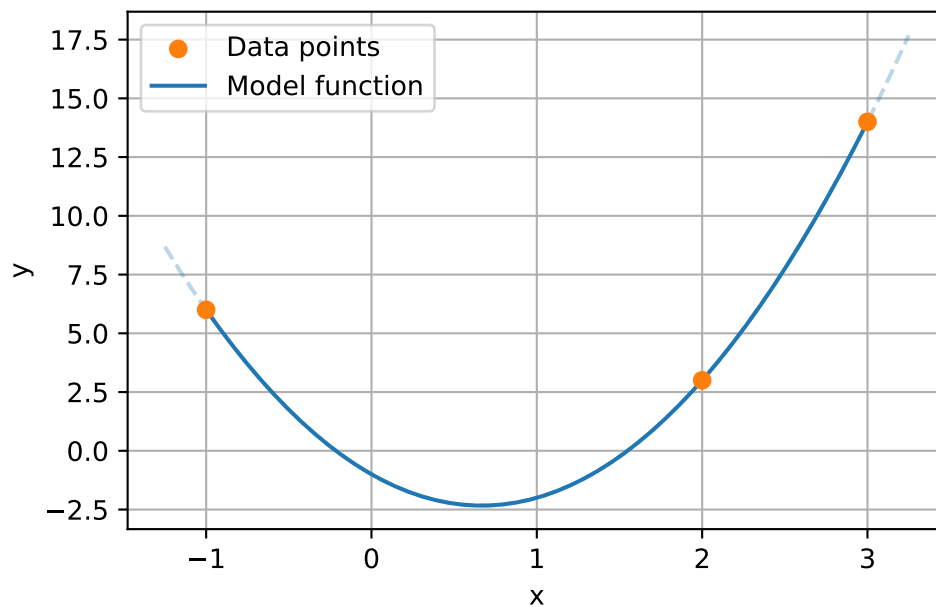
```

x_line = np.linspace(np.min(x)-dx, np.max(x)+dx, N)
plt.plot(x_line, func(x_line), '--', alpha=0.3, color='C0')

plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()

plt.show()

```



This can be generalized: n data points can be exactly represented by a polynomial of degree $(n - 1)$. Finding the appropriate coefficients is the Lagrange interpolation problem. For the desired polynomial $P(x)$, it holds that:

$$P(x_i) = y_i \quad i = 1, \dots, n$$

The existence and uniqueness of such a polynomial can be proven. The polynomial is given by:

$$P(x) = \sum_{i=1}^n y_i I_i(x)$$

$$\text{with } I_i(x) = \prod_{j=1, i \neq j}^n \frac{x - x_j}{x_i - x_j}$$

Alternatively, a system of equations defined by a polynomial basis $\phi_i(x)$ can be solved. For each of the n point pairs, we have:

$$y(x_i) = \sum_{i=1}^m \beta_i \cdot \phi_i(x_i) = \beta_1 \cdot \phi_1(x_i) + \dots + \beta_m \cdot \phi_m(x_i) = y_i$$

The general system of equations is

$$\begin{pmatrix} \phi_1(x_1) & \phi_2(x_1) & \dots & \phi_m(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \dots & \phi_m(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x_n) & \phi_2(x_n) & \dots & \phi_m(x_n) \end{pmatrix} \cdot \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

and using the polynomial basis

$$\underbrace{\begin{pmatrix} 1 & x_1 & \dots & x_1^{n-1} \\ 1 & x_2 & \dots & x_2^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \dots & x_n^{n-1} \end{pmatrix}}_V \cdot \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

The matrix V is called the [Vandermonde matrix](#) and can be solved exactly for $m = n$ if $x_i \neq x_j$ for all i, j with $i \neq j$.

In Python, the interpolation problem can be solved with the [function `numpy.polynomial.polynomial.polyfit\(\)`](#) which we can call after importing the module as `poly.polyfit()`. The following example demonstrates its use.

The data points in the example follow the function $f(x)$, which is only used to generate the data points:

$$f(x) = \frac{1}{2} + \frac{1}{1 + x^2}$$

First, we generate the data points.

```
def func(x):
    return 0.5 + 1/(1 + x**2)
```

```
xmin = -5
xmax = 5
x = np.linspace(xmin, xmax, 100)
y = func(x)
```

```
n = 5
xi = np.linspace(xmin, xmax, n)
yi = func(xi)
```

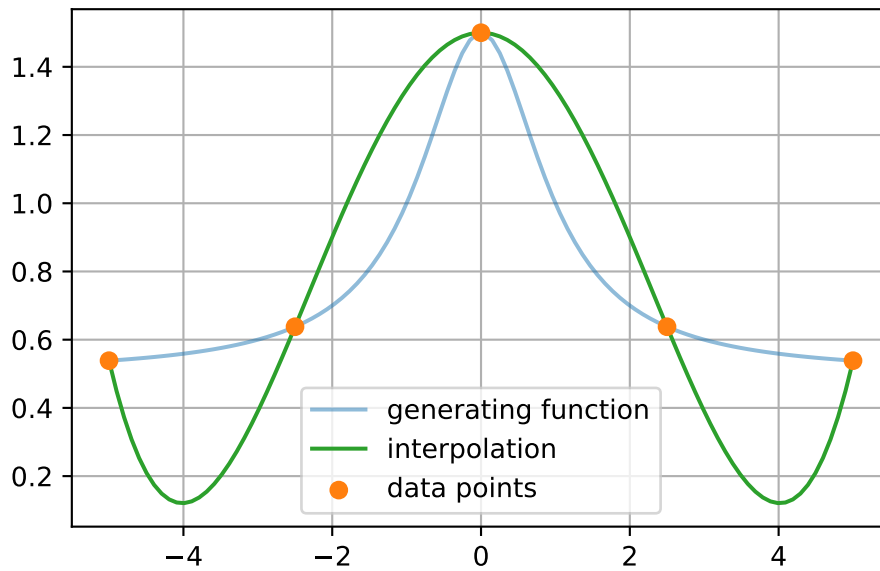
Now we perform the interpolation for 5 and 15 data points

```
P = poly.polyfit(xi, yi, n-1)
print("Interpolation coefficients:")
print(P)
```

```
Interpolation coefficients:
[ 1.50000000e+00 -1.12346671e-16 -1.71087533e-01  6.98020740e-18
  5.30503979e-03]
```

```
plt.plot(x, y, color='C0', alpha=0.5, label='generating function')
plt.plot(x, poly.polyval(x, P), color='C2', label='interpolation')
plt.scatter(xi, yi, color='C1', label='data points', zorder=3)
plt.legend()
plt.grid()

plt.show()
```

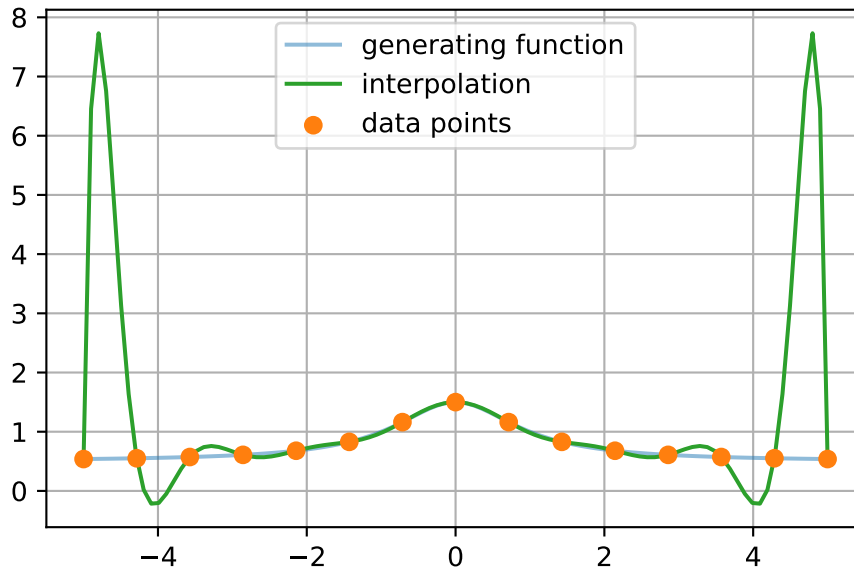


```
n = 15
xi = np.linspace(xmin, xmax, n)
yi = func(xi)

P = poly.polyfit(xi, yi, n-1)

plt.plot(x, y, color='C0', alpha=0.5, label='generating function')
plt.plot(x, poly.polyval(x, P), color='C2', label='interpolation')
plt.scatter(xi, yi, color='C1', label='data points', zorder=3)
plt.legend()
plt.grid()

plt.show()
```



Interpolation always satisfies the required condition $y(x_i) = y_i$. However, high-degree polynomials often lead to unrealistic results. Large oscillations occur, which become more pronounced as the degree increases.

3 Fitting

Fitting involves finding a model function that does not necessarily match the measured data exactly. If a polynomial is used, its degree is much smaller than the number of data points. Linear regression is an example of fitting with a polynomial of degree one.

To fit using a polynomial, you can use the function `numpy.polynomial.polynomial.polyfit()`, just like with polynomial interpolation, but this time with a smaller polynomial degree.

In the following example, model data are first generated and then fitted with appropriate polynomials.

```
import numpy as np
import matplotlib.pyplot as plt

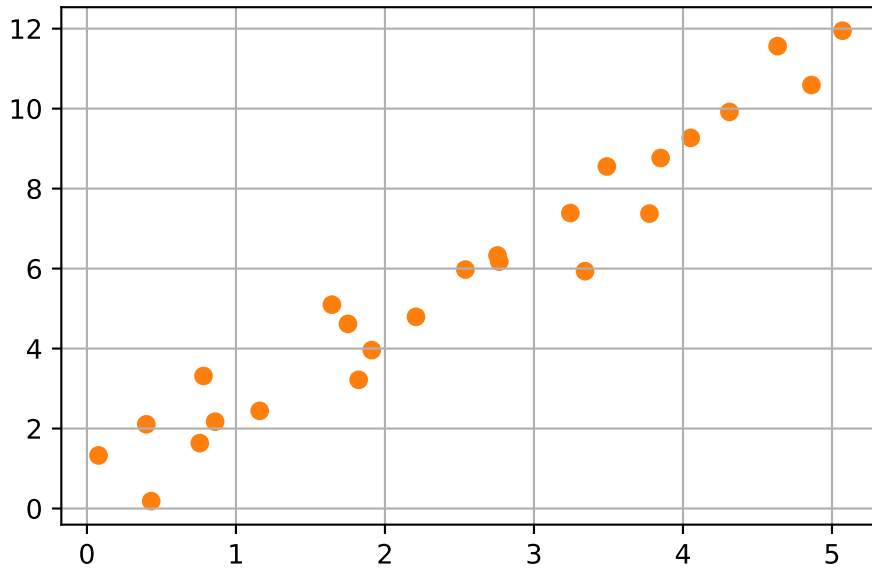
xmin = 0
xmax = 5
x = np.linspace(xmin, xmax, 100)

ni = 25 # number of data points

# x-values with slight noise
xi = np.linspace(xmin, xmax, ni) + 0.2 * (2 * np.random.random(ni) - 1)

# y(x) = 2*x + 0.5 with slight noise
yi = 2 * xi + 0.5 + 2 * (2 * np.random.random(ni) - 1)

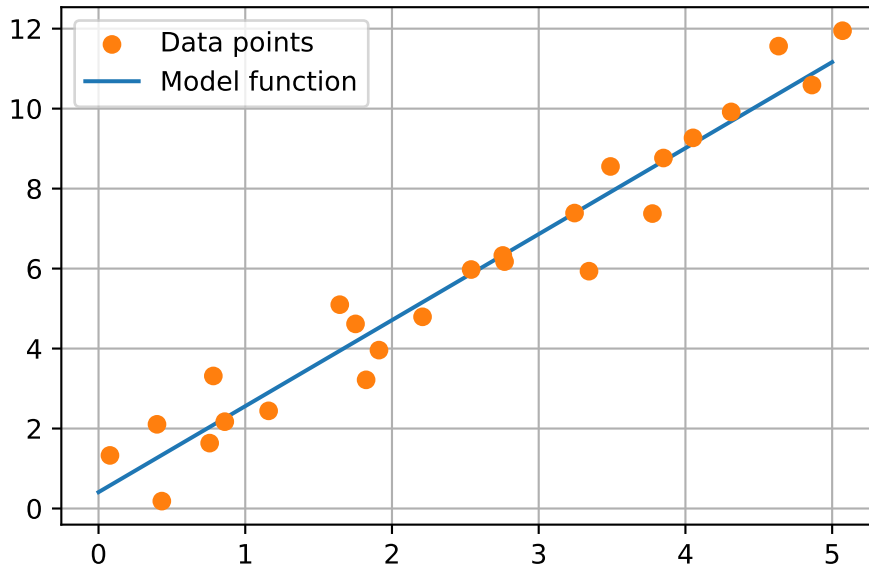
plt.scatter(xi, yi, color='C1')
plt.grid()
plt.show()
```



```
P1 = poly.polyfit(xi, yi, 1)

plt.scatter(xi, yi, color='C1', zorder=3, label='Data points')
plt.plot(x, poly.polyval(x, P1), color='C0', label="Model function")
plt.grid()
plt.legend()

plt.show()
```



In this example, a first-degree polynomial and a second-degree polynomial are determined.

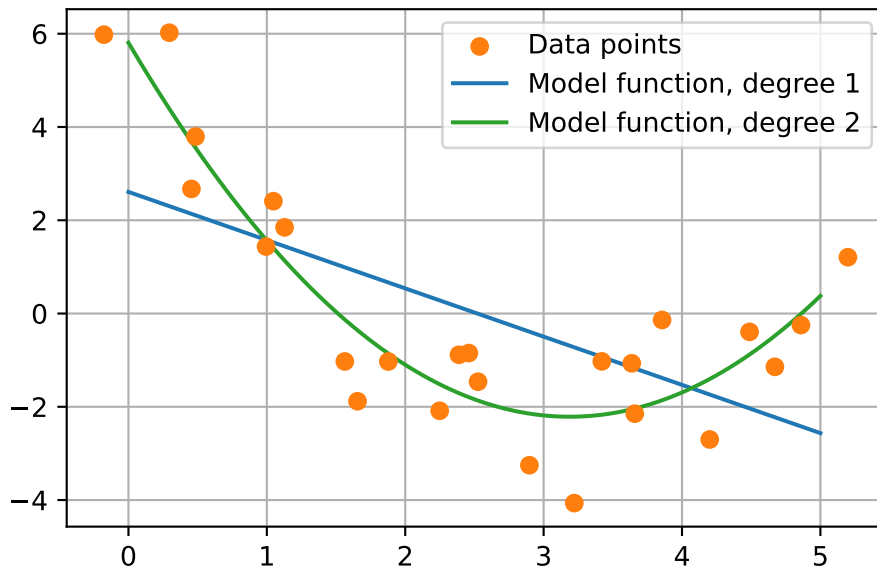
```
# x-values with slight noise
xi = np.linspace(xmin, xmax, ni) + 0.2*(2 * np.random.random(ni) - 1)

# y(x) = (x-2)^2 - 2x + 2.5 with slight noise
yi = (xi - 2)**2 - 2*xi + 2.5 + 2*(2 * np.random.random(ni) - 1)

P1 = poly.polyfit(xi, yi, 1)
P2 = poly.polyfit(xi, yi, 2)

plt.scatter(xi, yi, color='C1', zorder=3, label='Data points')
plt.plot(x, poly.polyval(x, P1), color='C0', label="Model function, degree 1")
plt.plot(x, poly.polyval(x, P2), color='C2', label="Model function, degree 2")
plt.grid()
plt.legend()

plt.show()
```



4 Splines

Polynomial interpolation attempts to find a global model function. However, high-degree polynomials are generally not suitable for interpolating many points. A different approach is taken by splines, which use multiple low-degree polynomials to interpolate many points. The polynomials typically have degrees between one and three.

4.1 Definition

For $n + 1$ data points (x_i, y_i) , a spline function s_k , here a polynomial spline, can be defined as follows:

- It is assumed that the data points are sorted, i.e., $x_0 < x_1 < \dots < x_n$.
- For each $i = 0 \dots n - 1$, s_k is a polynomial of degree k on the interval $[x_i, x_{i+1}]$.
- s_k is $(k - 1)$ -times continuously differentiable on $[x_0, x_n]$.

Examples:

- $k = 1$: piecewise linear spline
- $k = 3$: cubic polynomial splines (B-splines)

4.2 Cubic Splines

The cubic polynomial splines s_3 ($k = 3$) commonly used in practice have the following properties:

- $s_3|_{[x_i, x_{i+1}]} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$
- s_3 is twice continuously differentiable on $[x_0, x_n]$, especially at the support points x_i of the data points.

The coefficients β_i are determined as follows:

- From the $n + 1$ data points, there are n intervals, i.e., with four coefficients each, there are a total of $4n$ coefficients.
- Exact representation of the data points ($n + 1$ equations), i.e., $s_3(x_i) = y_i$.

- Smoothness conditions at the internal data points ($i = 1 \dots n - 1$), each providing $(n - 1)$ equations):

$$\begin{aligned}s_3'(x_i)_- &= s_3'(x_i)_+ \\ s_3''(x_i)_- &= s_3''(x_i)_+ \\ s_3'''(x_i)_- &= s_3'''(x_i)_+\end{aligned}$$

- This gives $4n - 2$ equations for $4n$ coefficients.

To determine the two remaining equations, boundary or endpoint conditions are required. The most common conditions are:

- Natural splines: the curvature at the boundary vanishes, i.e.:

$$s_3''(x_0) = s_3''(x_n) = 0$$

- Periodic splines: the slope and curvature are equal at both boundaries:

$$\begin{aligned}s_3'(x_0) &= s_3'(x_n) \\ s_3''(x_0) &= s_3''(x_n)\end{aligned}$$

- Hermite splines: the slopes at the boundaries are explicitly specified (here by u and v):

$$\begin{aligned}s_3'(x_0) &= u \\ s_3'(x_n) &= v\end{aligned}$$

4.3 Application

The following shows two examples, s_1 and s_3 , for creating splines in Python.

```
# Generate data points
n = 7
xi = np.linspace(0, np.pi, n)
yi = np.sin(xi)
```

For s_1 splines, the function `np.interp()` can be used. It performs linear interpolation between given pairs of values.

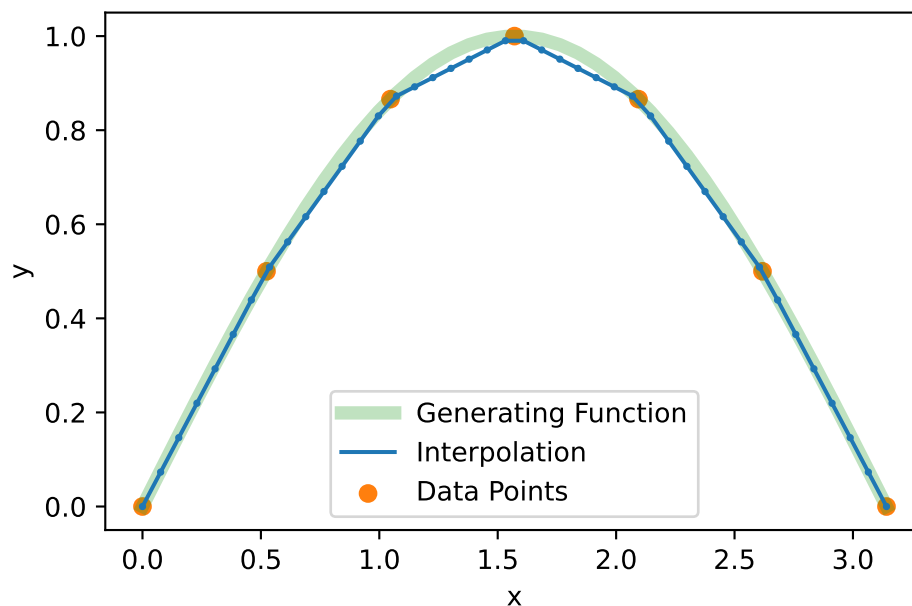
```
# Range of values for visualizing the interpolation
x = np.linspace(0, np.pi, n*6)
y = np.sin(x)
```

```
# Interpolation
y_s1 = np.interp(x, xi, yi)

plt.plot(x, y, alpha=0.3, color='C2', lw=5,
label='Generating Function')
plt.plot(x, y_s1, color='C0', label='Interpolation')
plt.scatter(x, y_s1, s=3, zorder=3, color='C0')
plt.scatter(xi, yi, color='C1', label='Data Points')

plt.xlabel('x')
plt.ylabel('y')
plt.legend();

plt.show()
```



The s_3 splines can be computed using functions from the `scipy` module. First, the coefficients are determined (`scipy.interpolate.splrep`), which then allow the desired evaluation using the function `scipy.interpolate.splev`.

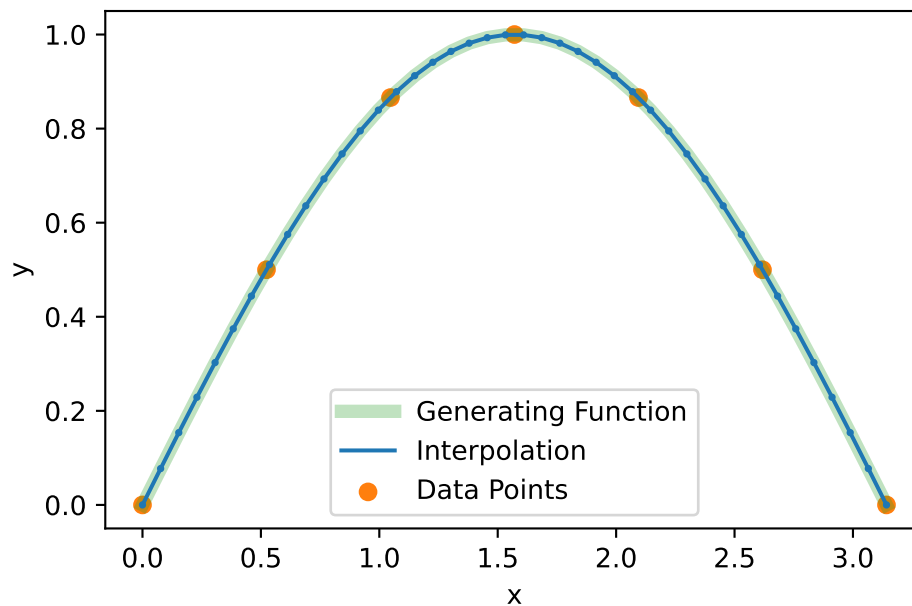
```
import scipy.interpolate as si
```

```
s3 = si.splrep(xi, yi)
y_s3 = si.splev(x, s3)
```

```
plt.plot(x, y, alpha=0.3, color='C2', lw=5,
label='Generating Function')
plt.plot(x, y_s3, color='C0', label='Interpolation')
plt.scatter(x, y_s3, s=3, zorder=3, color='C0')
plt.scatter(xi, yi, color='C1', label='Data Points')

plt.xlabel('x')
plt.ylabel('y')
plt.legend();

plt.show()
```



5 Trend Smoothing – Reducing Noise

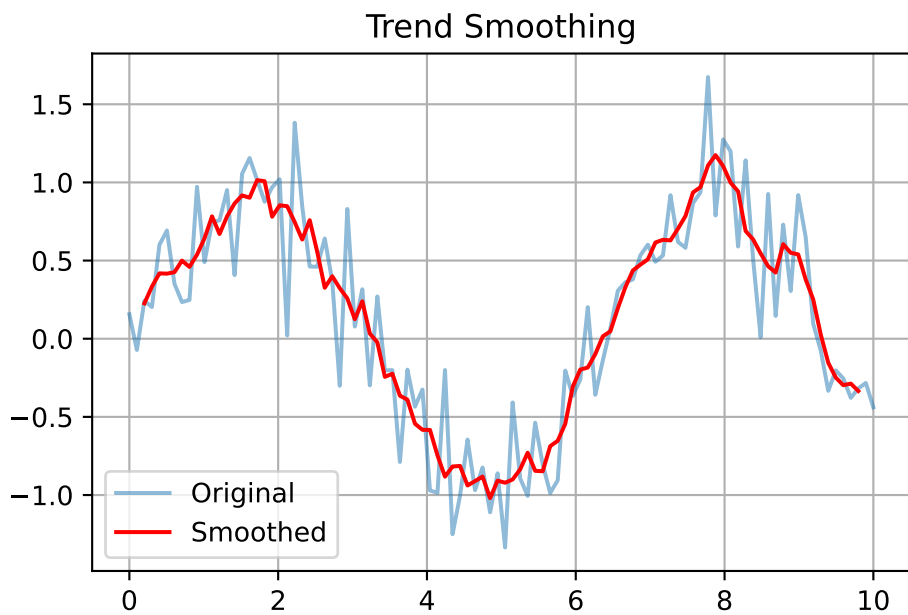
Noisy data? A **moving average** smooths curves:

```
data = np.genfromtxt("01-daten/trenddaten-mit-rauschen.csv", delimiter=",", skip_header=1)
x = data[:, 0]
y = data[:, 1]

window = 5
weights = np.ones(window) / window
y_smooth = np.convolve(y, weights, mode='valid')

plt.plot(x, y, label="Original", alpha=0.5)
plt.plot(x[(window-1)//2:-(window//2)], y_smooth, label="Smoothed", color='red')
plt.legend()
plt.grid(True)
plt.title("Trend Smoothing")

plt.show()
```



6 Exercises

6.1 Exercise: Analyzing Balloon Flight Data

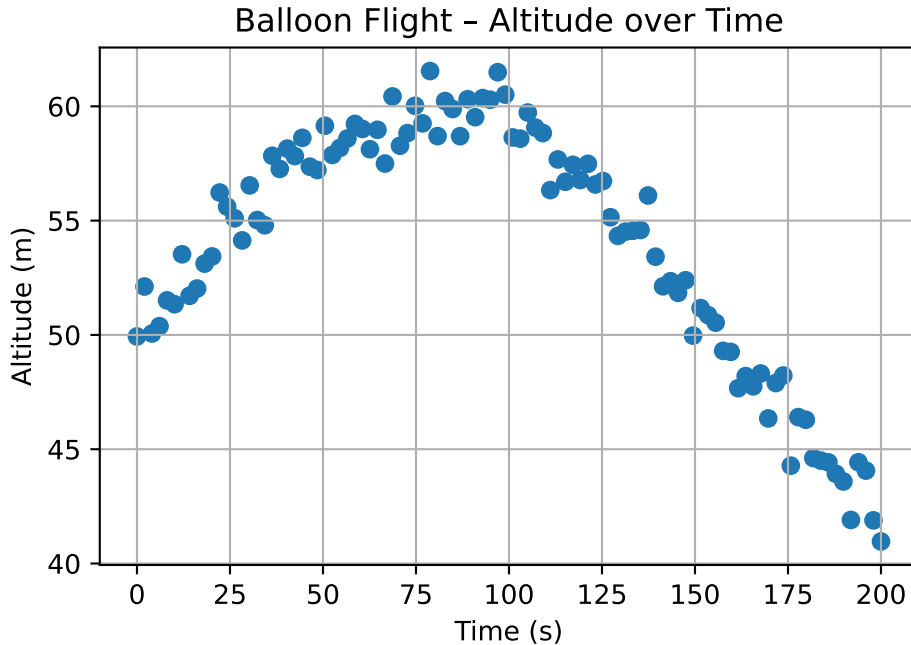
The file '01-daten/messdaten-ballonfahrt.txt' contains measurement data for the altitude of a balloon during a flight. The first line of the file lists the measured quantities and their units.

```
# Read file header as a string
balloon = np.genfromtxt("01-daten/messdaten-ballonfahrt.txt", delimiter=",", max_rows=4, dtype=object)

print("The header of the file:")
print(balloon)
```

```
The header of the file:
[['Zeit (s)' 'Hoehe (m)']
 ['0.0' '49.93175839467537']
 ['2.02020202020203' '52.11727320291297']
 ['4.0404040404041' '50.0624468191306']]
```

Let us take a look at the data:



We are looking for a polynomial that describes the ascent rate, i.e., the rate of change of height over time, of the hot air balloon.

Write a program that:

- reads the height from the data and calculates the ascent rate,
- fits the ascent rate with a third-degree polynomial.

Plot the height, the ascent rate, and the fit each against time. Label the axes.

💡 Tipp 1: Sample Solution: Balloon Flight

First, read in the data.

```
# Read data
balloon = np.genfromtxt("01-daten/messdaten-ballonfahrt.txt", delimiter=";", skip_header=1)
time = balloon[:, 0]
height = balloon[:, 1]
```

Next, we calculate the ascent rate $\frac{\Delta H_{he}}{\Delta Z_{eit}}$. For this, we use the function `np.diff()`. It computes the difference of each value to its predecessor: `result[i] = value[i+1] - value[i]`. Therefore, no difference is calculated for the *i*-th value, and the result is one element shorter. The ascent rate in meters per second is obtained by dividing the two

sequences.

```
# Calculate differences

delta_height = np.diff(height)
print("Change in height: ", delta_height[0:4])

delta_time = np.diff(time)
print("Change in time: ", delta_time[0:4])

# Calculate ascent rate

ascent_rate = delta_height / delta_time
print("Ascent rate: ", ascent_rate[0:4])
```

```
Change in height: [ 2.18551481 -2.05482638  0.32026988  1.12796595]
Change in time:  [2.02020202 2.02020202 2.02020202 2.02020202]
Ascent rate:     [ 1.08182983 -1.01713906  0.15853359  0.55834314]
```

Using `numpy.polynomial.polynomial.polyfit()`, we first fit a third-degree polynomial, then use `numpy.polynomial.polynomial.polyval()` to evaluate the fitted data.

```
poly3_ascent_rate = poly.polyfit(time[:-1], ascent_rate, deg = 3)

fit_ascent_rate = poly.polyval(x = time[:-1], c = poly3_ascent_rate)
```

```

# plotting

plt.suptitle("Balloon Flight")

## subplot: altitude over time

plt.subplot(1, 3, 1)
plt.scatter(time, altitude)
plt.title("Altitude")
plt.xlabel("Time (s)")
plt.ylabel("Altitude (m)")
plt.grid(True)

## subplot: ascent rate over time

## length of the ascent_rate array is n-1

plt.subplot(1, 3, 2)
plt.scatter(time[:-1], ascent_rate)
plt.title("Ascent Rate")
plt.xlabel("Time (s)")
plt.ylabel("Ascent Rate (m/s)")
plt.grid(True)

## subplot: cubic polynomial fitted ascent rate over time

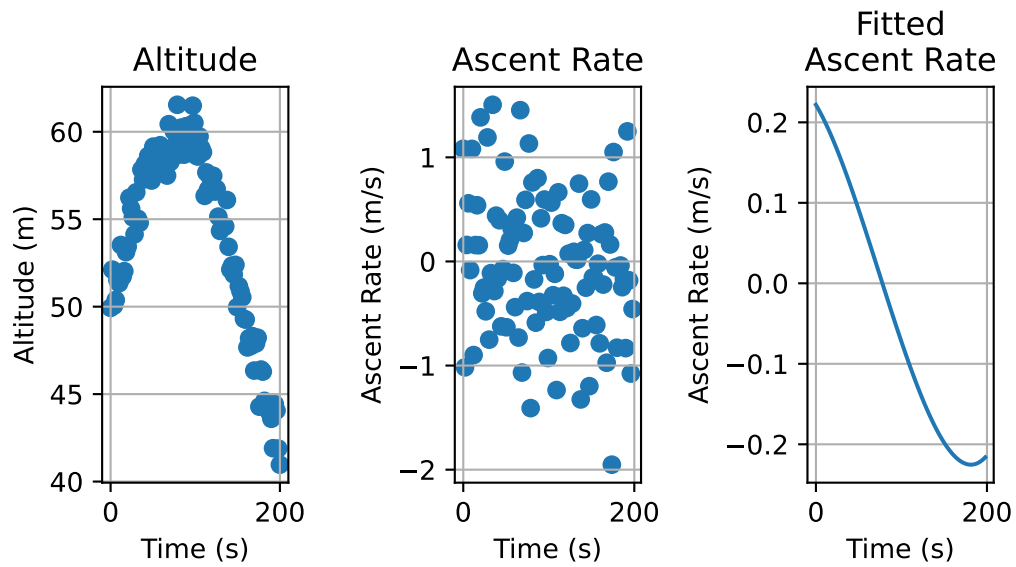
## length of the ascent_rate array is n-1

plt.subplot(1, 3, 3)
plt.plot(time[:-1], fit_ascent_rate)
plt.title("Fitted\nAscent Rate")
plt.xlabel("Time (s)")
plt.ylabel("Ascent Rate (m/s)")
plt.grid(True)

plt.tight_layout()
plt.show()

```

Balloon Flight



6.2 Exercise: Beam Deflection in Civil Engineering

A beam is loaded at its center.

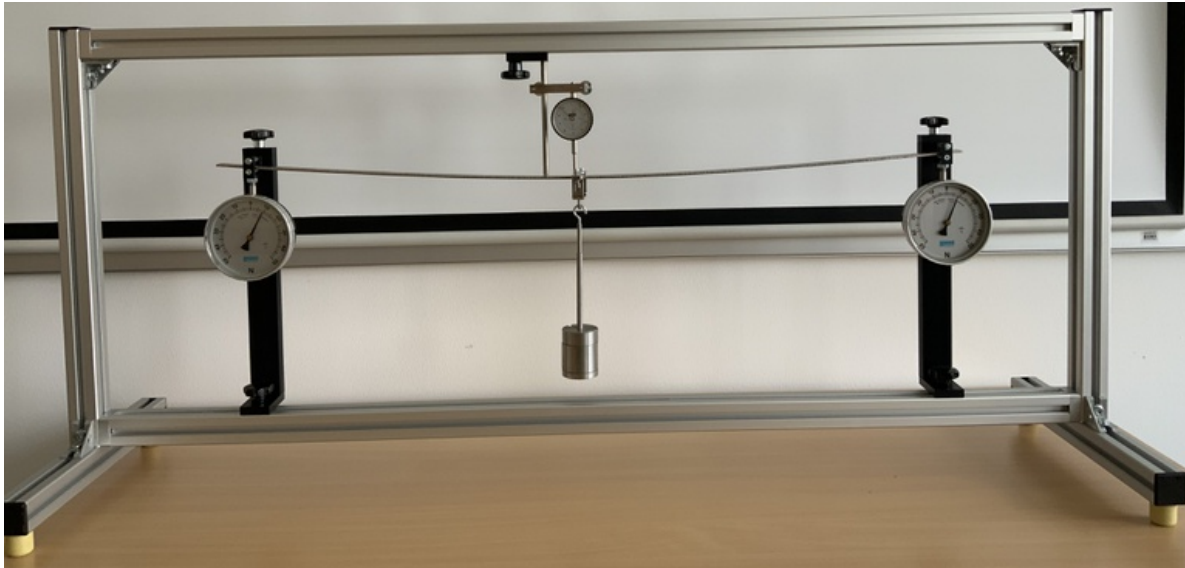


Abbildung 6.1: Example of a beam deflection experiment setup

Beam deflection from the University of the Bundeswehr Munich. The work is available on the website of [UniBw M](#). No year specified.

Let's take a look at the structure of the data.

```
beam = np.genfromtxt("01-daten/balken-durchbiegung.csv", delimiter=",", max_rows=4, dtype='s')

print("The header of the file:")
print(beam)
```

The header of the file:

```
[['Position (m)' 'Durchbiegung (mm)']
 ['0.0' '-0.28820719619532215']
 ['0.20408163265306123' '-0.22193122300286544']
 ['0.40816326530612246' '0.013978362500916705']]
```

The deflection is measured at 50 points. Smooth the data and display both the measured and smoothed data together.

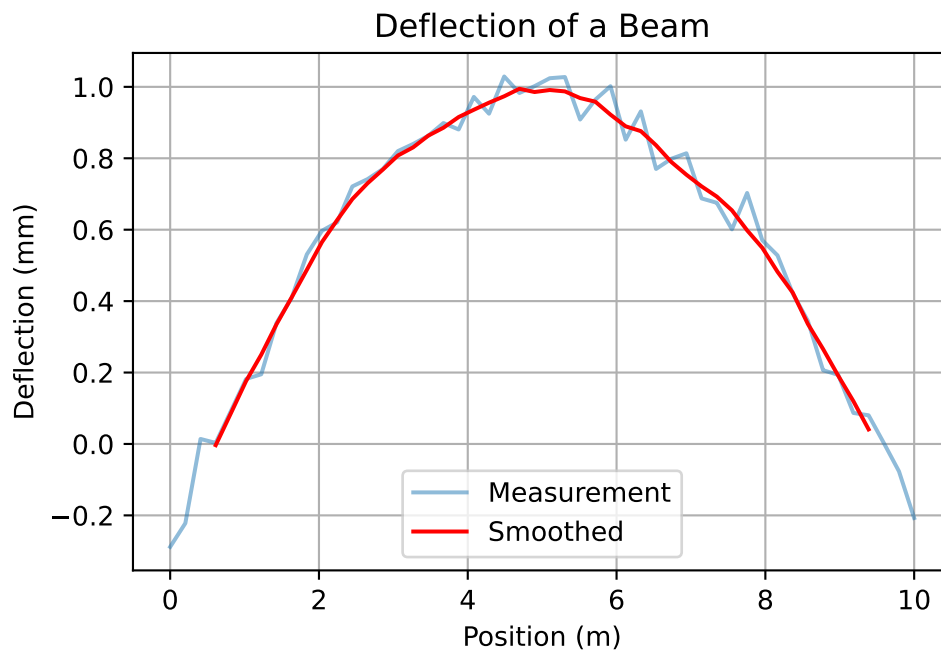
💡 Tipp 2: Example Solution Beam

```
beam = np.genfromtxt("01-daten/balken-durchbiegung.csv", delimiter=";", skip_header=1)
x = beam[:, 0]
y = beam[:, 1]

window = 7
weights = np.ones(window) / window
y_smooth = np.convolve(y, weights, mode='valid')

plt.plot(x, y, label="Measurement", alpha=0.5)
plt.plot(x[(window-1)//2:-((window//2)]), y_smooth, label="Smoothed", color='red')
plt.title("Deflection of a Beam")
plt.xlabel("Position (m)")
plt.ylabel("Deflection (mm)")
plt.legend()
plt.grid(True)

plt.show()
```



6.3 Exercise: Neutron Scattering

The file '01-daten/neutronen.txt' contains data from a neutron scattering experiment. The file header:

```
# Read file header as a string
neutrons = np.genfromtxt("01-daten/neutronen.txt", delimiter="\t", max_rows=4, dtype='str')
print(neutrons)
```

```
[['E(MeV)' 'sigma(mb)' 'Delta sigma(mb)']
 ['0.00' '10.60' ' 1.39']
 ['25.00' '16.00' ' 2.09']
 ['50.00' '45.00' ' 3.85']]
```

In the column 'E(MeV)', the energy of a neutron fired at a target is listed in mega-electronvolts. In the column 'sigma(mb)', the cross section, measured in millibarns (mb), is listed. In the column 'Delta sigma(mb)', the uncertainty of the measured cross section, also in millibarns (mb), is listed.

1. Read in the data
2. Try to fit the first two columns with a polynomial of a suitable degree and display the result graphically.
3. Fit the data again, but this time use splines.
4. Display the result graphically as well and compare it with the polynomial fit.
5. What are the advantages and disadvantages of each method?

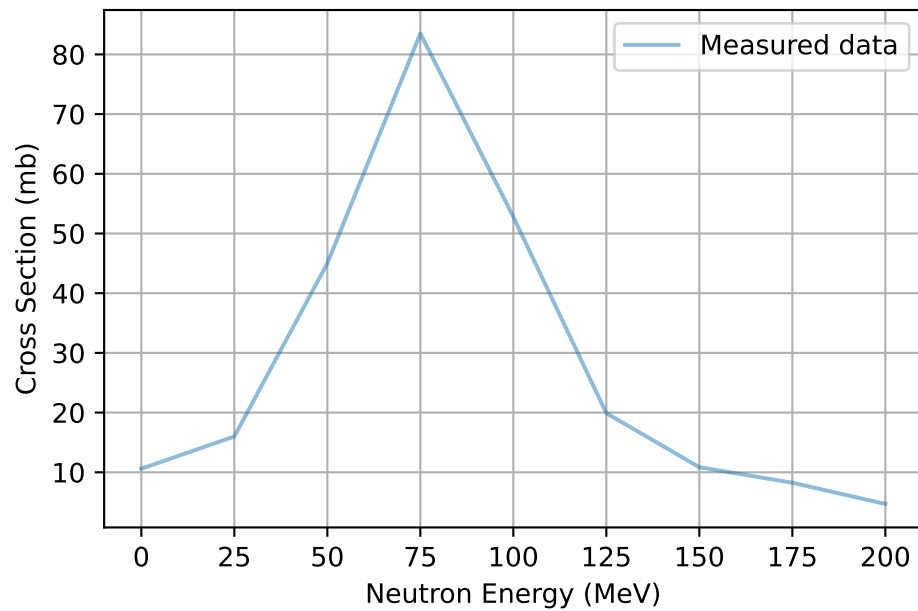
💡 Tipp 3: Sample Solution: Neutron Scattering

1. Read in the data

```
neutrons = np.genfromtxt("01-daten/neutronen.txt", delimiter="\t", skip_header=1)
neutron_energy = neutrons[:, 0]
cross_section = neutrons[:, 1]

plt.plot(neutron_energy, cross_section, label="Measured data", alpha=0.5)
plt.xlabel("Neutron Energy (MeV)")
plt.ylabel("Cross Section (mb)")
plt.legend()
plt.grid(True)

plt.show()
```



2. • 3. Polynomial and Spline Fitting

```
# Polynomial

polynomial5 = poly.polyfit(neutron_energy, cross_section, deg=5)
fit = poly.polyval(x=neutron_energy, c=polynomial5)

# Splines with SciPy

spline3 = si.splrep(neutron_energy, cross_section)
y_spline3 = si.splev(neutron_energy, spline3)

# Splines with NumPy

x_new = np.linspace(neutron_energy.min(), neutron_energy.max(), num=50)
y_spline1 = np.interp(x_new, neutron_energy, cross_section) # linear interpolation
```

4. Graphical Representation

```

plt.figure(figsize=[7, 5])

# Comparison with cubic splines

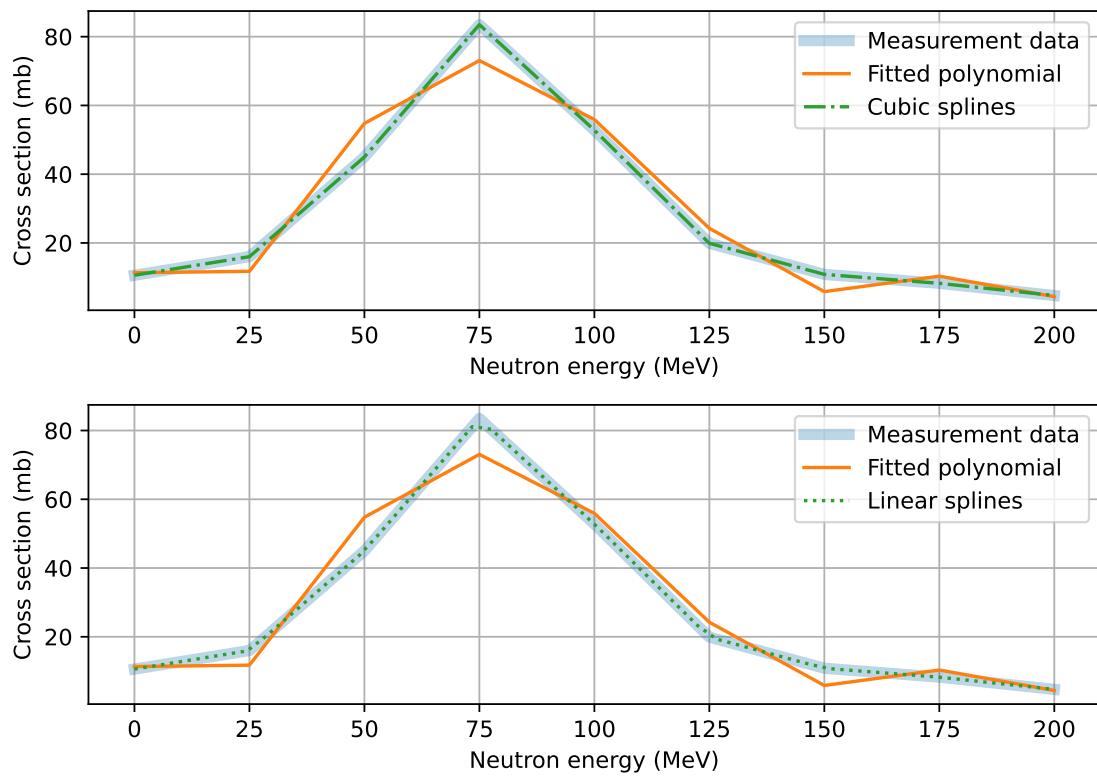
plt.subplot(2, 1, 1)
plt.plot(neutron_energy, cross_section, linewidth=5, alpha=0.3, label="Measurement data")
plt.plot(neutron_energy, fit, label="Fitted polynomial")
plt.plot(neutron_energy, y_spline3, linestyle='dashdot', label="Cubic splines")
plt.xlabel("Neutron energy (MeV)")
plt.ylabel("Cross section (mb)")
plt.legend()
plt.grid(True)

# Comparison with linear splines

plt.subplot(2, 1, 2)
plt.plot(neutron_energy, cross_section, linewidth=5, alpha=0.3, label="Measurement data")
plt.plot(neutron_energy, fit, label="Fitted polynomial")
plt.plot(x_new, y_spline1, linestyle='dotted', label="Linear splines")
plt.xlabel("Neutron energy (MeV)")
plt.ylabel("Cross section (mb)")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

```



5.: The splines match the data exactly. This can be an advantage if no smoothing of the data is desired. The polynomial, on the other hand, is not fitted exactly to the data. However, all data points can be approximated using a single model equation.

Quellen

Baitsch, Matthias (2019). *Vorlesungsskript Mathematik B (Master) Stochastik*.